

گزارش یادگیری هفته چهارم: تست نویسی، احراز هویت و امنیت وب

مقدمه

در هفته چهارم بوت کمپ، تمرکز اصلی بر سه رکن اساسی توسعه نرم افزار حرفه ای بود: تست نویسی برای اطمینان از صحت عملکرد، احراز هویت برای تشخیص هویت کاربران، و امنیت وب برای محافظت از داده ها و سیستم. این مباحث، مرز میان یک برنامه نویس مبتدی و یک توسعه دهنده حرفه ای را مشخص می کنند. در ادامه، هر یک از این مفاهیم را به صورت کامل و بدون کد بررسی می کنیم.

۱. تست نویسی (Testing)

۱.۱ هرم تست

در مهندسی نرم افزار، تست ها بر اساس سطح و دامنه پوشش به سه دسته اصلی تقسیم می شوند که به صورت یک هرم تصور می شوند:

- تست واحد (Unit Test): قاعده هرم که بیشترین تعداد تست ها را تشکیل می دهد. در این سطح، کوچک ترین بخش های کد (معمولاً یک تابع یا متد) به صورت کاملاً ایزوله و بدون وابستگی به پایگاه داده، شبکه یا فایل تست می شوند. این تست ها بسیار سریع (در حد میلی ثانیه) اجرا می شوند و باید برای هر ورودی، خروجی یکسان و قابل پیش بینی داشته باشند.
- تست یکپارچگی (Integration Test): لایه میانی هرم که تعامل چند بخش مختلف سیستم را با هم می آزماید. برخلاف تست واحد، در اینجا ممکن است از یک پایگاه داده واقعی (اما تستی) استفاده شود و جریان داده بین اجزا بررسی گردد. این تست ها کندتر از تست واحد هستند اما سناریوهای واقعی تری را پوشش می دهند.

- تست انتها به انتها (EndtoEnd Test): رأس هرم با کمترین تعداد تست. در این سطح، کل جریان یک کاربر واقعی از ابتدا تا انتها شبیه سازی می شود؛ مثلاً باز کردن مرورگر، ثبت نام، ورود و انجام یک عملیات کامل. این تست ها کندترین و پیچیده ترین هستند اما اطمینان می دهند که کل سیستم به درستی کار می کند.

۱.۲ انواع دیگر تست

- علاوه بر سه سطح اصلی، انواع دیگری از تست نیز در فرآیند توسعه نرم افزار استفاده می شوند:
- تست API: بررسی می کند که واسط برنامه نویسی (endpointها) در برابر درخواست های HTTP پاسخ صحیح (از نظر کد وضعیت، ساختار پاسخ و محتوا) برمی گردانند.
- تست عملکرد (Performance Test): میزان توان عملیاتی سیستم تحت بار سنگین را می سنجد (مانند تست های Locust که در هفته های قبل انجام دادیم).
- تست امنیت (Security Test): به دنبال یافتن آسیب پذیری هایی مانند تزریق SQL یا نشت داده است.
- تست رگرسیون (Regression Test): اطمینان می دهد که تغییرات جدید کد، قابلیت های قبلی را خراب نکرده است.
- تست دود (Smoke Test): یک بررسی سریع و سطحی برای اطمینان از اینکه بخش های حیاتی سیستم پس از استقرار جدید، هنوز کار می کنند.

۱.۳ پوشش تست (Test Coverage)

پوشش تست معیاری برای سنجش این است که چه میزان از کد منبع توسط تست ها اجرا شده است. این معیار معمولاً به صورت درصد بیان می شود و انواع مختلفی دارد: پوشش خط (Line Coverage) که نشان می دهد چند درصد از خطوط کد در حین تست ها اجرا شده اند، و پوشش شاخه (Branch Coverage) که بررسی می کند آیا تمام مسیرهای ممکن در ساختارهای شرطی (if/else) پیمایش شده اند یا خیر. هدف معمول دستیابی به پوشش بالای هشتاد درصد است، هرچند پوشش صد درصدی همیشه ضروری یا ممکن نیست.

۱.۴ ابزارهای Mocking

در تست واحد، برای ایزوله کردن کامل کد تحت تست، وابستگی‌های خارجی آن (مانند اتصال به پایگاه داده یا فراخوانی یک API خارجی) با اشیاء جعلی (Mock) جایگزین می‌شوند. این کار علاوه بر افزایش سرعت تست، اطمینان می‌دهد که شکست یک تست صرفاً به دلیل خطا در خود آن واحد کد است، نه یک سرویس خارجی.

۲. احراز هویت (Authentication)

۲.۱ تفاوت Authorization و Authentication

درک تمایز این دو مفهوم حیاتی است:

- احراز هویت (Authentication): پاسخ به این سوال که "تو چه کسی هستی؟". فرآیند تأیید هویت کاربر (مثلاً از طریق نام کاربری و رمز عبور).
- مجوزدهی (Authorization): پاسخ به این سوال که "تو چه کارهایی می‌توانی انجام دهی؟". تعیین سطح دسترسی کاربر پس از احراز هویت (مثلاً یک کاربر عادی نمی‌تواند کاربران دیگر را حذف کند).

۲.۲ روش‌های احراز هویت

- Basic Authentication: ساده‌ترین روش که در آن نام کاربری و رمز عبور به صورت کدگذاری شده (Base64) در سرآیند (Header) هر درخواست ارسال می‌شود. به دلیل سادگی، پیاده‌سازی آن آسان است اما معایب امنیتی جدی دارد؛ از جمله اینکه در هر درخواست اطلاعات حساس ارسال می‌شود و استفاده از آن بدون پروتکل HTTPS بسیار خطرناک است. این روش امروزه صرفاً برای ابزارهای داخلی یا توسعه مناسب است.
- TokenBased Authentication: در این روش، پس از یک بار ورود موفق، سرور یک نشانه (Token) برای کاربر صادر می‌کند. کاربر این نشانه را ذخیره کرده و در سرآیند درخواست‌های بعدی خود ضمیمه می‌کند. معروف‌ترین نوع این نشانه‌ها، نشانه وب JSON یا JWT است. JWT یک رشته خودتوصیفگر است که از سه بخش تشکیل شده: سرآیند (نوع الگوریتم

رمزنگاری)، بدنه (اطلاعات کاربر مانند شناسه و زمان انقضا) و امضا (برای اطمینان از عدم دستکاری). مزیت بزرگ JWT این است که سرور نیازی به ذخیره‌سازی وضعیت نشست (Session) ندارد و با بررسی امضای نشانه می‌تواند هویت کاربر را تأیید کند.

- OAuth 2.0: یک پروتکل استاندارد برای واگذاری دسترسی محدود، بدون به اشتراک‌گذاری رمز عبور. متداول‌ترین کاربرد آن، ورود از طریق حساب‌های گوگل یا گیت‌هاب است. در این سناریو، کاربر به وب‌سایت مورد نظر (Client) اجازه می‌دهد تا به اطلاعات مشخصی (مانند نام و ایمیل) از حساب گوگل او دسترسی پیدا کند، بدون اینکه رمز عبور گوگل خود را در اختیار آن وب‌سایت قرار دهد.

- Cookie/Session Authentication: روش سنتی در وب‌سایت‌های مبتنی بر HTML. پس از ورود کاربر، سرور یک شناسه نشست (Session ID) ایجاد کرده و در یک کوکی (Cookie) در مرورگر کاربر ذخیره می‌کند. مرورگر به طور خودکار این کوکی را با هر درخواست به سرور بازمی‌گرداند و سرور با بررسی آن، کاربر را شناسایی می‌کند. برخلاف JWT، این روش "حالت‌دار" (Stateful) است و سرور باید اطلاعات نشست را در حافظه یا پایگاه داده نگهداری کند.

۳. امنیت وب و OWASP

۳.۱ OWASP چیست؟

OWASP (پروژه باز امنیت برنامه‌های وب) یک بنیاد غیرانتفاعی است که هر چند سال یکبار فهرستی از ۱۰ آسیب‌پذیری بحرانی امنیتی در برنامه‌های وب را منتشر می‌کند. این فهرست، مرجعی استاندارد برای توسعه‌دهندگان و متخصصان امنیت است.

۳.۲ مرور مهم‌ترین موارد OWASP

- کنترل دسترسی شکسته (Broken Access Control): زمانی رخ می‌دهد که کاربران می‌توانند با حدس زدن یا دستکاری شناسه‌های موجود در URL یا درخواست، به داده‌هایی

دسترسی پیدا کنند که متعلق به آنها نیست. راه حل، بررسی مداوم مالکیت داده ها در سمت سرور است.

- شکست های رمزنگاری (Cryptographic Failures): به محافظت نادرست از داده های حساس اشاره دارد. ارسال اطلاعات با HTTP به جای HTTPS، ذخیره سازی رمز عبور به صورت متن ساده یا با الگوریتم های شکسته شده مانند MD5، یا استفاده از کلیدهای رمزنگاری قدیمی از مصادیق آن هستند.

- تزریق (Injection): زمانی رخ می دهد که داده های ورودی کاربر به عنوان بخشی از یک دستور (مانند کوئری SQL، فرمان سیستم عامل یا درخواست LDAP) تفسیر شود. مهاجم می تواند با ارسال ورودی های دستکاری شده، دستورات دلخواه خود را اجرا کند. معروف ترین نوع آن SQL Injection است. راه حل قطعی، استفاده از کوئری های پارامتری است که در آن داده های ورودی کاربر هرگز با ساختار کوئری ترکیب نمی شوند.

- طراحی ناامن (Insecure Design): به آسیب پذیری هایی اشاره دارد که ناشی از طراحی نادرست معماری نرم افزار است، نه صرفاً خطاهای کدنویسی. برای مثال، طراحی یک سیستم احراز هویت که سوالات بازیابی امنیتی ضعیفی دارد.

- پیکربندی نادرست امنیتی (Security Misconfiguration): شامل هرگونه تنظیمات پیش فرض ناامن، فعال بودن ویژگی های غیرضروری، یا نمایش اطلاعات خطای بیش از حد (مانند ردپای کامل خطا) به کاربران نهایی می شود. یک وبسایت حرفه ای هرگز جزئیات فنی خطاهای خود را در محیط تولید به کاربران نمایش نمی دهد.

- شناسایی و احراز هویت شکسته (Identification and Authentication Failures): ضعف هایی مانند اجازه استفاده از رمزهای عبور ساده و متداول، عدم محدودیت در تعداد تلاش های ناموفق ورود (که منجر به حملات جستجوی فراگیر می شود)، یا عدم باطل سازی صحیح نشانه های احراز هویت پس از خروج کاربر.

۴. توابع هش (Hash Functions)

۴.۱ مفهوم و کاربردها

تابع هش الگوریتمی است که یک ورودی با طول دلخواه را دریافت کرده و به یک خروجی با طول ثابت و ظاهراً تصادفی تبدیل می‌کند. این فرآیند یک‌طرفه است؛ یعنی محاسبه خروجی از روی ورودی آسان، اما بازسازی ورودی از روی خروجی عملاً غیرممکن است. از هش برای تأیید یکپارچگی فایل‌ها (Checksum)، شناسه‌گذاری در سیستم‌های گیت و بلاکچین، و مهم‌تر از همه، ذخیره‌سازی امن رمزهای عبور استفاده می‌شود.

۴.۲ ضعف الگوریتم‌های سریع برای رمز عبور

الگوریتم‌های عمومی مانند MD5 یا SHA256 به دلیل سرعت بسیار بالا برای هش کردن رمزهای عبور مناسب نیستند. سرعت بالای آنها به مهاجم اجازه می‌دهد میلیاردها حدس را در هر ثانیه آزمایش کند (حمله جستجوی فراگیر). همچنین در برابر حملات "جدول رنگین‌کمان" (Rainbow Table) که پایگاه‌های داده‌ای عظیم از هش‌های از پیش محاسبه‌شده هستند، آسیب‌پذیرند.

۴.۳ Salt و کندی؛ دفاع مدرن

برای مقابله با حملات فوق، دو تکنیک به کار می‌رود:

Salt (نمک): یک رشته تصادفی یکتا که پیش از هش کردن به رمز عبور هر کاربر اضافه می‌شود. این کار تضمین می‌کند که دو کاربر با رمز عبور یکسان، هش‌های کاملاً متفاوتی در پایگاه داده داشته باشند و جداول رنگین‌کمان را بی‌اثر می‌کند.

هش کند: الگوریتم‌های مدرن مانند Argon2، bcrypt و scrypt عمداً طوری طراحی شده‌اند که کند باشند و منابع محاسباتی زیادی مصرف کنند. به این ترتیب، هر تلاش برای حدس رمز عبور زمان‌بر و پرهزینه می‌شود و حملات جستجوی فراگیر را عملاً غیرممکن می‌سازند. bcrypt به صورت پیش‌فرض یک Salt تصادفی تولید کرده و آن را در خروجی نهایی ذخیره می‌کند.

جمع‌بندی

هفته چهارم ما را با مفاهیم بنیادین تضمین کیفیت و امنیت نرم‌افزار آشنا کرد. فرا گرفتیم که تست‌نویسی صرفاً یک فعالیت جانبی نیست، بلکه بخشی جدایی‌ناپذیر از فرآیند توسعه است که در سطوح مختلف (از یک تابع ساده تا کل جریان کاربر) انجام می‌شود. با روش‌های مختلف احراز هویت و مزایا و معایب هر یک آشنا شدیم و فهمیدیم که انتخاب روش مناسب به سناریوی کاربردی بستگی دارد. در نهایت، با بررسی ۱۰ آسیب‌پذیری برتر OWASP و الگوریتم‌های هش مدرن، درک کردیم که امنیت یک لایه اضافی نیست، بلکه باید در تار و پود طراحی و کدنویسی تنیده شود. این دانش، پایه و اساس ساخت نرم‌افزارهای حرفه‌ای، مقیاس‌پذیر و قابل اعتماد است.